1. Project Title

AI-Powered Healthcare Monitoring Agent using RAG and Multi-Tool Agentic Workflow

2. Problem Statement and Background Study

Healthcare monitoring for individuals is often fragmented, reactive, and dependent on manual tracking. Patients struggle with:

- Medication adherence and scheduling
- Understanding drug interactions
- Monitoring health metrics over time
- Accessing reliable medical information
- Identifying potential health risks early

Most existing systems are either static dashboards or rule-based reminder apps. They lack intelligent reasoning, contextual understanding, and dynamic decision-making.

There is a need for an **LLM-based AI agent** capable of:

- Understanding natural language health queries
- Retrieving reliable medical knowledge
- Dynamically calling tools (e.g., medication checker, risk predictor)
- Generating personalized health insights
- Maintaining health history and context

This project aims to build an **agentic AI healthcare assistant**, not a simple chatbot, that follows a structured reasoning workflow using Retrieval-Augmented Generation (RAG) and tool calling.

3. Solution Overview

We propose an AI-powered Healthcare Monitoring Agent that integrates:

- Large Language Model (LLM) reasoning
- Retrieval-Augmented Generation (RAG)
- Multi-tool decision making
- Health data storage
- Predictive analytics

The agent will:

1. Receive user health queries.
2. Analyze intent using an LLM.
3. Decide whether to:
    - Retrieve medical knowledge (RAG),
    - Call a tool (e.g., medication reminder, interaction checker),
    - Access stored patient data.
4. Generate an accurate, contextualized response.
5. Store updated information for future interactions.

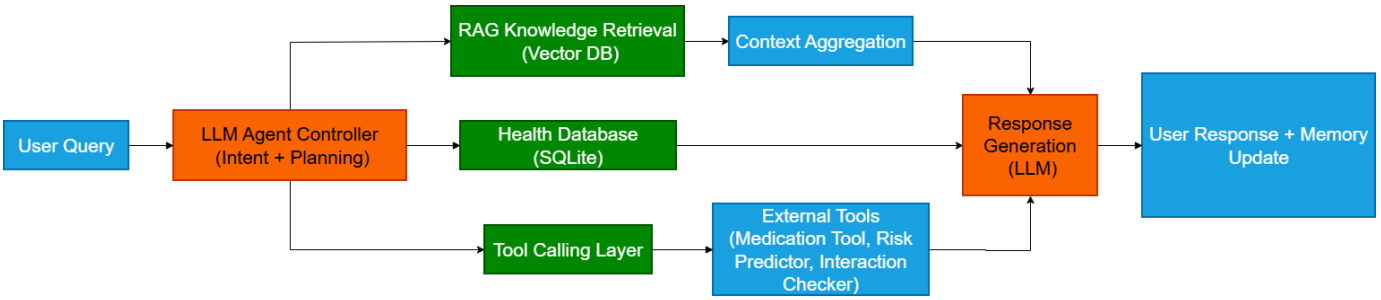
The system will be built in two phases:

- **Phase 1 (Track A):** Core AI agent with RAG + tool calling + minimal interface.
- **Phase 2 (Track B):** Full production-ready architecture with API backend, role-based access, and dashboard.

The primary focus is **AI agent quality and workflow design**.

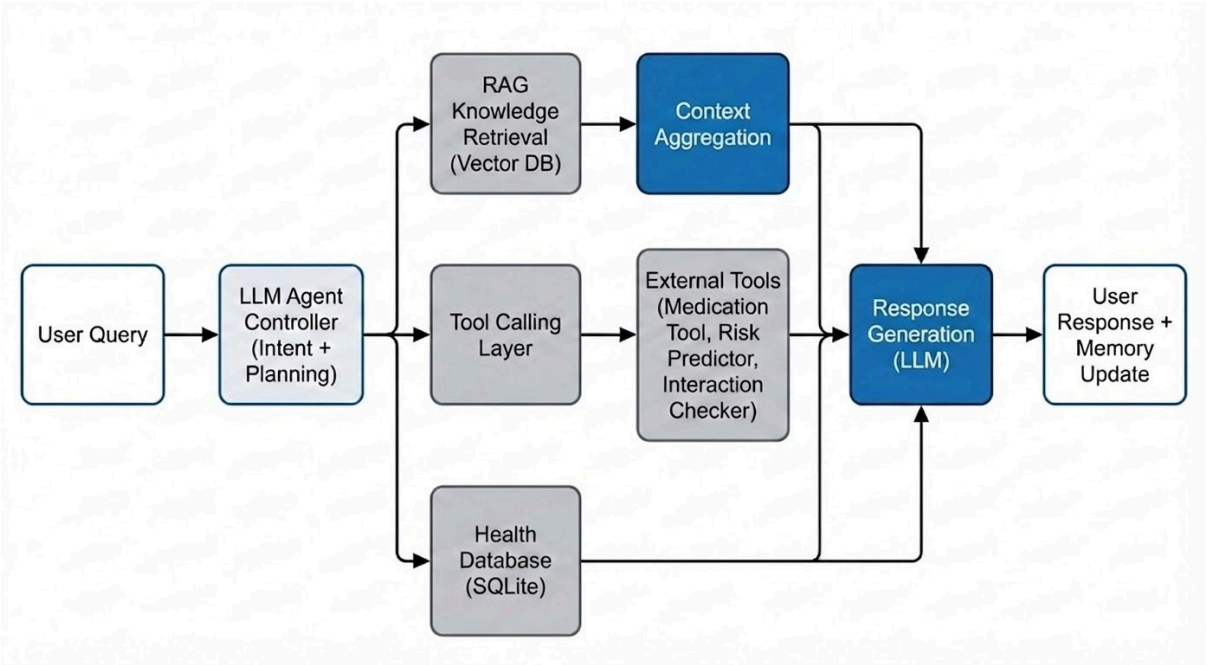
4. Agent Workflow Diagram

The AI agent follows an agentic decision pipeline rather than a linear chatbot flow as depicted in the below flowchart (built using draw.io).



Flowchart Using Draw.io

Other Flow:



5. System Architecture

High-Level Architecture

The system consists of five main layers:

1. Interface Layer

- Gradio-based UI (Track A)
- React dashboard (Track B)

2. Agent Layer

- LangChain Agent
- LLM (HuggingFace model)
- Conversation memory
- Tool selection logic

3. RAG Layer

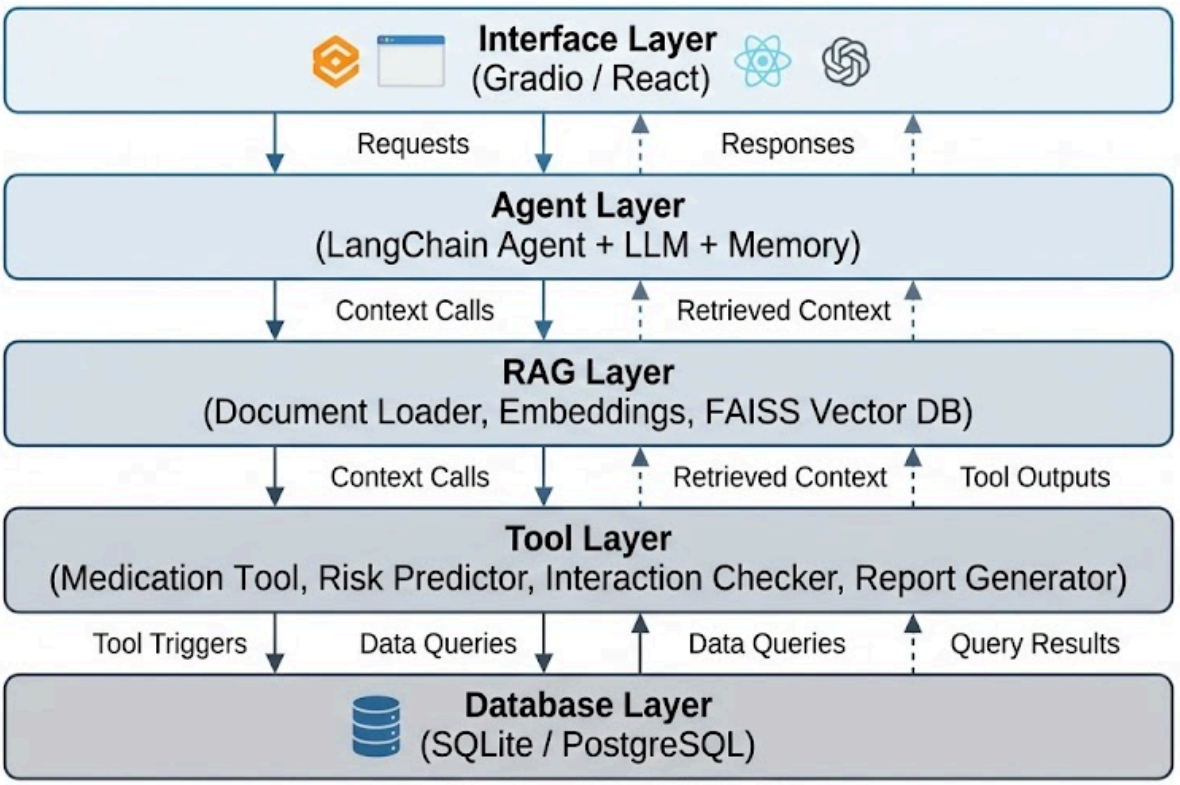
- Document Loader
- Text SplitterEmbedding
- ModelFAISS
- Vector Database
- Retriever

4. Tool Layer

- Medication Reminder Tool
- Drug Interaction Checker
- Health Risk Predictor
- Report Generator

5. Data Layer

- SQLite (Track A)
- PostgreSQL + Redis(if needed) (Track B)



Planned System Architecture for Healthcare Monitoring Application

6. Tool List

The AI agent will dynamically call the following tools:

1. Medication Reminder Tool

- Stores medication schedule
- Sends reminders
- Tracks adherence
- Updates medication logs

2. Drug Interaction Checker

- Takes multiple medications as input
- Checks for known interactions
- Flags high-risk combinations
- Returns safety recommendations

3. Health Risk Predictor

- Inputs: age, BP, sugar level, BMI, etc.
- Uses ML model (scikit-learn)
- Predicts health risk category
- Provides preventive recommendations

4. Health Report Generator

- Aggregates patient history
- Generates structured health summary
- Highlights anomalies
- Suggests improvements

These tools are invoked dynamically by the LLM agent.

7. RAG Plan

The system will implement a Retrieval-Augmented Generation pipeline.

Knowledge Sources

- Drug information datasets
- Medical guidelines (public datasets)
- Nutrition and lifestyle information
- Disease descriptions

RAG Pipeline Steps

1. Document Loading
2. Text Chunking
3. Embedding Generation (Sentence Transformers)
4. FAISS Vector Index Creation
5. Query Embedding
6. Top-K Retrieval
7. Context Injection into LLM

Vector Database

- FAISS (Track A)
- Optional upgrade to managed vector DB (Track B)

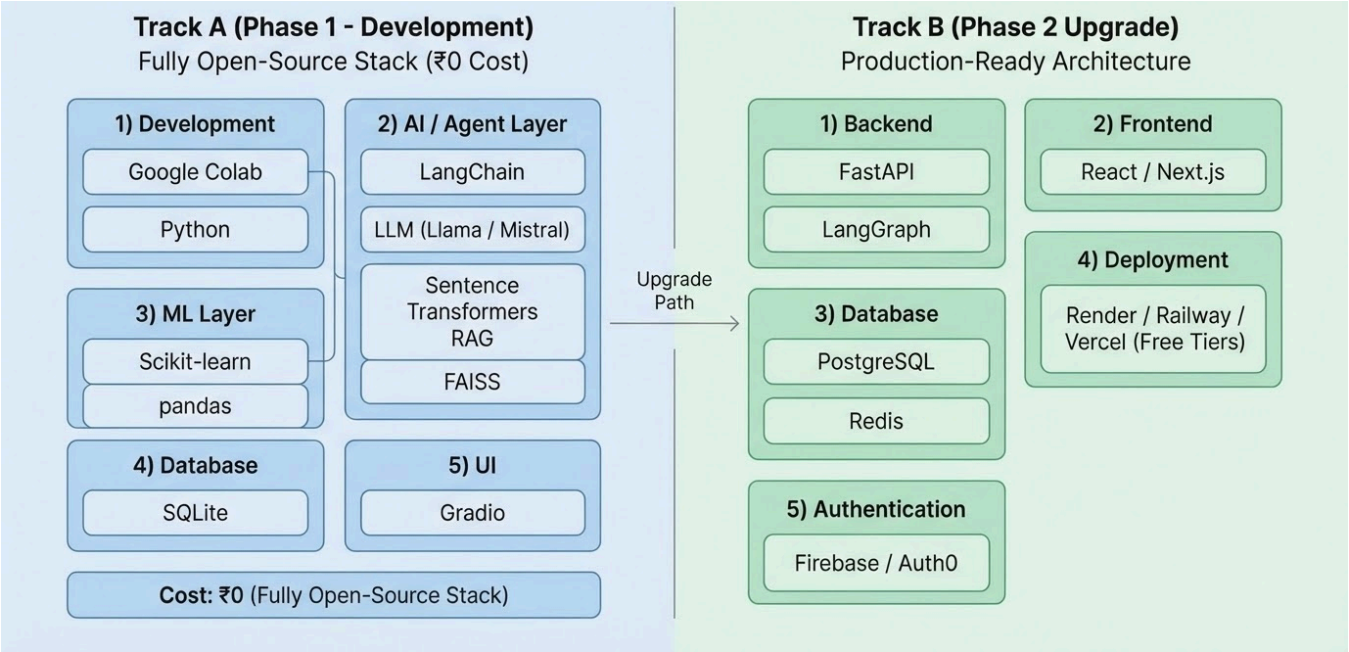
Embedding Model

- Sentence Transformers (HuggingFace)

The RAG system ensures accurate, citation-based health responses.

8. Tech Stack

Track A (Phase 1)	Track B (Phase 2 Upgrade)
Development ● Google Colab ● Python **AI / Agent** ● LangChain ● HuggingFace LLM (Llama/Mistral) ● Sentence Transformers **RAG** ● FAISS **ML** ● Scikit-learn ● pandas **Database** ● SQLite **UI** ● Gradio **Cost** ₹0 (Fully open-source stack)	**Backend** ● FastAPI ● LangGraph **Frontend** ● React / Next.js **Database** ● PostgreSQL ● Redis **Deployment** ● Render / Railway / Vercel (free tiers) **Authentication** ● Firebase/Auth0



Tools Used for Healthcare Monitoring Application

9. Weekly Roadmap

Week 1

- Finalize architecture
- Define tools
- Design RAG plan
- Create workflow diagram
- Setup GitHub repository

Week 2

- Implement RAG pipeline
- Build vector DB
- Test retrieval accuracy

Week 3

- Develop tools
- Implement medication + interaction + risk tools

Week 4

- Integrate tools with agent
- Add memory
- Build working prototype

Week 5–6

- Multi-agent refinement
- Add predictive analytics
- Improve evaluation metrics
- Begin VS Code transition

Week 7

- FastAPI backend
- React dashboard

- Role-based structure

Week 8

- Final integration
- Deployment
- Demo preparation
- Documentation

10. GitHub Link

GitHub Repository (Public):
Avi007-debug/Team_Agent_Wars_Healthcare-Monitoring-AI-Agent

The repository will contain:

- Project structure
- Architecture diagrams
- RAG implementation
- Tool implementations
- Agent orchestration code
- Documentation
- Weekly progress updates